

CSA1712

ARTIFICIAL INTELLIGENCE

NAME: VARSHINI M

REG NO.: 192511215

1. Hospital Doctor Shift Assignment using Backtracking Search

Problem Understanding

A hospital needs to assign three doctors (**D1, D2, D3**) to three shifts (**Morning, Afternoon, Night**) such that all given constraints are satisfied. Each doctor must be assigned exactly one shift, and each shift must have only one doctor.

Variables

- D1 = Doctor 1
- D2 = Doctor 2
- D3 = Doctor 3

Domain

Each doctor can initially work in any one of the following shifts:

- Morning
- Afternoon
- Night

Constraints

1. D1 cannot work the Night shift.
2. D2 must work before D3 (Morning < Afternoon < Night).
3. Only one doctor can be assigned to each shift.
4. D3 cannot work the Morning shift.
5. Every doctor must be assigned exactly one shift.

Constraint Analysis

Doctor Possible Shifts

D1 Morning, Afternoon

D2 Morning, Afternoon, Night

D3 Afternoon, Night

Backtracking Search

Step 1

Assign D1 = Morning

Constraint Check:

- D1 is not assigned Night. ✓

Current Assignment:

- D1 → Morning

Step 2

Assign D2 = Afternoon

Constraint Check:

- Shift available ✓

Current Assignment:

- D1 → Morning
- D2 → Afternoon

Step 3

Assign D3 = Night

Constraint Check:

- D3 is not Morning ✓
- D2 works before D3 ✓
- Only one doctor per shift ✓

All constraints are satisfied.

Search Tree

Start

|

D1 = Morning

|

D2 = Afternoon

|

D3 = Night

|

Solution Found

Constraint Verification

Constraint	Status
D1 $\neq$ Night	✓
D2 before D3	✓
One doctor per shift	✓
D3 $\neq$ Morning	✓
Every doctor assigned	✓

Final Assignment

Doctor Shift

D1 Morning

D2 Afternoon

D3 Night

Justification

Backtracking Search assigns one doctor at a time while checking constraints after every assignment. Since the first valid assignment satisfies all constraints, no further backtracking is required. The final assignment is complete, valid, and conflict-free.

2. Robot Path Finding using Breadth-First Search (BFS)

Problem Understanding

A robot operates in a 5×5 grid environment. It starts from **S** and must reach the goal **G**. Some cells contain obstacles. The robot can move only up, down, left, and right. Each movement has a cost of 1. Obstacles cannot be crossed. Manhattan Distance is used as the heuristic.

Constraints

1. Movement allowed only in four directions.
2. Cost of every movement is 1.
3. Obstacles cannot be crossed.
4. Manhattan Distance heuristic is used.

Example Grid

S . X . .

. . X . .

. . . X .

X

. . X . G

S = Start

G = Goal

X = Obstacle

. = Free Cell

BFS Node Expansion

Step 1

Queue:

[S]

Visit Start.

Step 2

Expand S

Possible moves:

- Right
- Down

Queue:

(1,2), (2,1)

Step 3

Expand (1,2)

Queue:

(2,1), (2,2)

Step 4

Expand (2,1)

Queue:

(2,2), (3,1)

Step 5

Expand (2,2)

Queue:

(3,1), (3,2)

Step 6

Expand (3,2)

Queue:

(3,3), (4,2)

Step 7

Expand (3,3)

Queue:

(4,2), (4,3)

Step 8

Expand (4,3)

Queue:

(4,4)

Step 9

Expand (4,4)

Queue:

(5,4)

Step 10

Expand (5,4)

Goal Found.

Priority Queue Updates

Step Queue

Start S

1 (1,2), (2,1)

2 (2,1), (2,2)

3 (2,2), (3,1)

4 (3,1), (3,2)

Step Queue

5 (3,3), (4,2)

6 (4,2), (4,3)

7 (4,4)

8 (5,4)

9 Goal

Manhattan Distance

Formula:

$$h = |x_1 - x_2| + |y_1 - y_2|$$

Example:

Current Node = (3,3)

Goal = (5,5)

$$h = |5 - 3| + |5 - 3|$$

$$h = 2 + 2 = 4$$

Optimal Path

S

↓

↓

→

→

↓

→

↓

→

G

Total Cost

Number of moves = 8

Cost per move = 1

Total Cost = **8**

Justification

Breadth-First Search explores nodes level by level and guarantees the shortest path when every movement has equal cost. Obstacles are avoided, and Manhattan Distance estimates the remaining distance to the goal. The final path is optimal with a total cost of 8.

3. Autonomous Rescue Robot in a Disaster-Affected Building

(a) Constraint Analysis

Problem Understanding

An autonomous rescue robot is deployed inside a disaster-affected building to locate and rescue survivors. The robot starts from **S** and must reach the survivor at **G**. Since the environment is partially observable, the robot has only limited information and must continuously update its knowledge while navigating safely.

Constraint 1: Robot can move only Up, Down, Left, and Right

- Movement is limited to four directions.
- Diagonal movement is not allowed.

Effect: Reduces possible actions and simplifies path planning.

Constraint 2: Obstacles cannot be crossed

Blocked cells represent debris or collapsed structures.

Effect: The robot must avoid blocked paths and search for alternate routes.

Constraint 3: Limited sensing capability

The robot can observe only adjacent cells.

Effect: The robot gradually discovers the environment and updates its internal map.

Constraint 4: Movement cost

- Normal move = 1
- Risky zone = 3 (1 + additional cost of 2)

Effect: The robot prefers safer paths that minimize total travel cost.

Constraint 5: Dynamic knowledge update

The robot continuously updates its knowledge while moving.

Effect: The robot replans its path whenever new obstacles or risky zones are discovered.

Summary of Constraints

Constraint	Effect
Limited movement	Only four directions allowed
Obstacles	Alternate paths required
Limited sensing	Map updated gradually
Movement cost	Chooses least-cost path
Dynamic updates	Replans when new information appears

(b) Solution Approach

Search Strategy

Uniform Cost Search (UCS) with Online Search

Why UCS?

- Handles different movement costs.
- Finds the minimum-cost path.
- Suitable for risky zones with additional costs.
- Guarantees an optimal solution.

Steps

1. Start at S.
2. Observe adjacent cells.
3. Update the internal map.
4. Insert valid neighboring cells into the priority queue.
5. Expand the node with the lowest accumulated cost.
6. If new obstacles are found, update the map and replan.
7. Continue until Goal G is reached.

Flowchart

Start

|

Initialize Robot

|

Observe Adjacent Cells

|

Update Map

|

Calculate Costs

|

Select Lowest-Cost Node

|

Move Robot

|
New Information?

| |
Yes No

| |
Update Continue

|
Goal Reached?

| |
No Yes

| |
Repeat Stop

(c) AI Concepts Applied

Intelligent Agent

The robot acts as an intelligent agent because it:

- Perceives the environment using sensors.
- Makes autonomous decisions.
- Learns new information.
- Chooses the best action to reach the goal.

Rationality

The robot behaves rationally by:

- Selecting actions that minimize travel cost.
- Avoiding obstacles.
- Avoiding risky zones when possible.
- Replanning whenever better paths are discovered.

Environment Type

Property	Description
Partially Observable	Cannot view the complete environment
Dynamic	Environment may change
Sequential	Current decisions affect future actions
Discrete	Building is represented as grid cells
Single Agent	Only one robot performs the rescue
Unknown	Complete map is unavailable initially

(d) Justification

The combination of **Uniform Cost Search (UCS)** and **Online Search** is the most suitable approach because it effectively handles all the given constraints. UCS guarantees the least-cost path by considering both normal movement costs and additional costs for risky zones. Online Search allows the robot to continuously update its knowledge as it discovers new obstacles or hazards. This enables dynamic replanning, ensuring safe and efficient navigation. Together, these techniques help the robot reach the survivor while minimizing travel cost and adapting to changes in the environment.

Conclusion

Constraint-Based Problem Solving enables intelligent systems to make effective decisions while satisfying predefined rules and limitations. In **Question 1**, Backtracking Search successfully assigns doctors to shifts while satisfying all constraints. In **Question 2**, Breadth-First Search finds the shortest path for the robot by systematically exploring the grid. In **Question 3**, the combination of Uniform Cost Search and Online Search allows an autonomous rescue robot to navigate a partially observable and dynamic environment safely while minimizing travel cost. These solutions demonstrate the practical application of AI search algorithms, intelligent agents, rational decision-making, and constraint satisfaction techniques in solving real-world problems.